

## LECTURE 14

# The Enhanced ER Model

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

# In This Lecture

- Why plain ER modeling breaks down for domains with natural sub-types
- Specialization: the top-down direction of reasoning
- Generalization: the bottom-up direction of reasoning
- Superclasses, subclasses, and inheritance of attributes AND relationships
- The four specialization/generalization constraints: disjoint vs. overlapping, total vs. partial
- Aggregation: treating a relationship itself as a participant in another relationship
- Composition: aggregation's stronger, ownership-implying cousin
- Categorization (union types): a subclass drawn from several unrelated superclasses

# Why Plain ER Isn't Enough

Plain ER (Lectures 12-13) is entities, relationships, and attributes, all at the same flat level. It breaks down specifically when a domain has natural sub-types sharing a common core but each adding its own extra structure:

## Attribute overload

Cram every subtype's attributes onto one entity type. Every SalesPersonnel-only attribute sits NULL on every Manager row and vice versa — a schema-level admission the entity type hides two+ different kinds of thing.

## Lost commonality

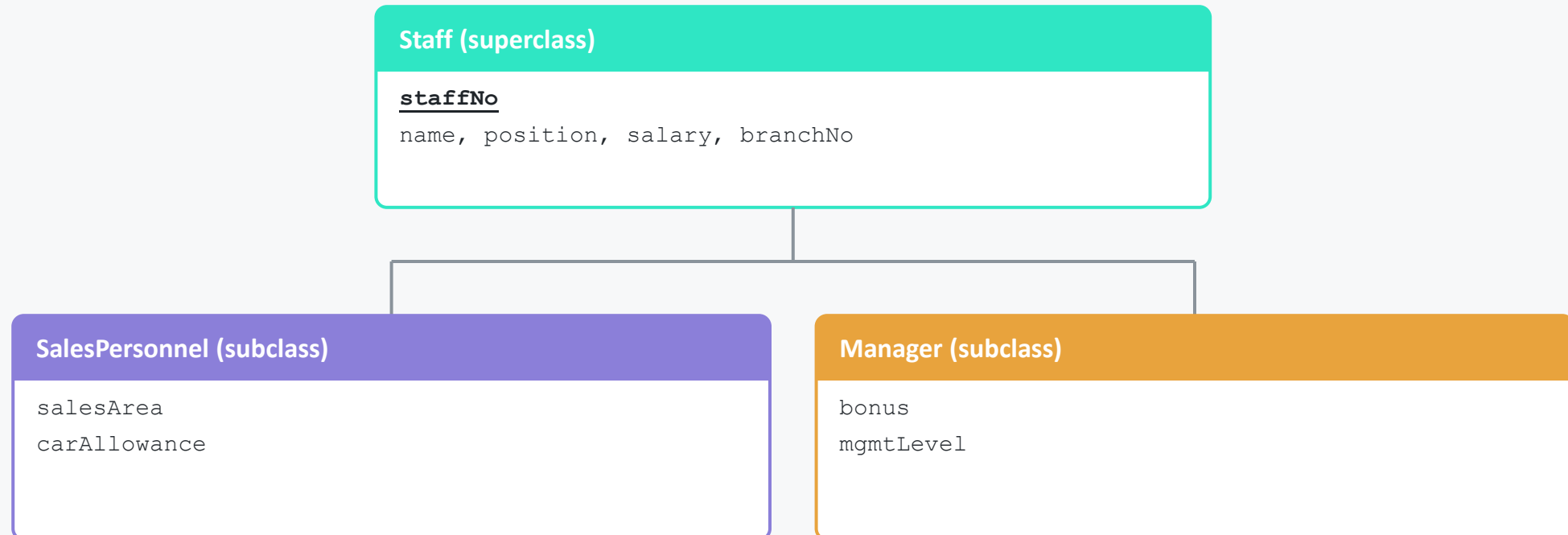
Model SalesPersonnel and Manager as two entirely separate entity types. staffNo/name/position/salary are duplicated in both, with no way to express “these are the same kind of thing” or query “all staff” without a manual union.

## The EER model's superclass/subclass machinery solves both at once

Shared attributes and relationships live ONCE, on the superclass, and each subclass adds only what's genuinely specific to it. This lecture adds four new modeling concepts on top of Lectures 12-13: specialization, generalization, aggregation, and categorization.

# Specialization — Top-Down

Start from ONE superclass already known to the model (Staff), and identify meaningful, distinct subgroupings based on a distinguishing characteristic (job role).

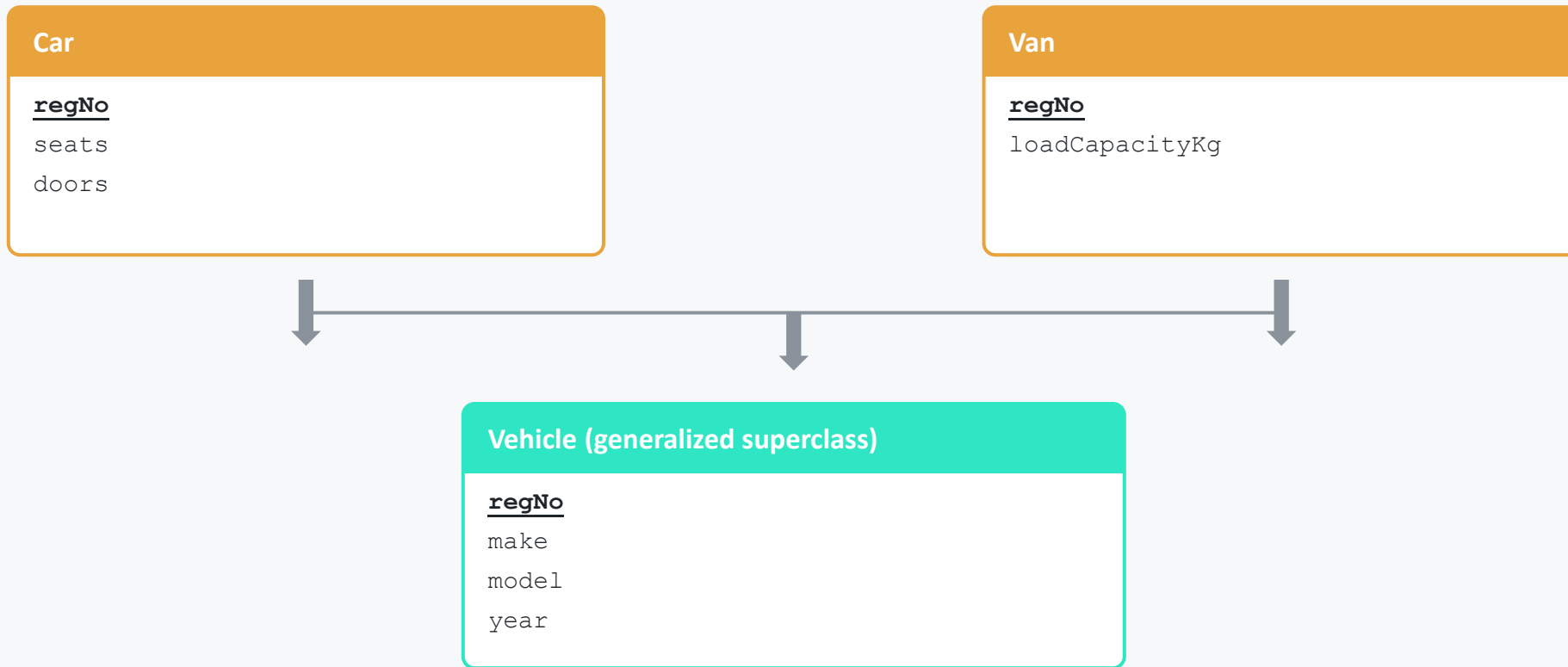


## The designer's reasoning

"I already have Staff. Within it, sales personnel and managers need extra, DIFFERENT attributes I have nowhere good to put — let me specialize Staff into subclasses that each carry only their own extra attributes."

# Generalization — Bottom-Up

Start from several already-known, separately modeled entity types that share a meaningful common core, and factor that shared core out into a new superclass.



“I already separately modeled Car and Van, each with regNo/make/model/year duplicated — let me generalize the shared attributes upward into a new Vehicle superclass.”

# Superclasses, Subclasses, and Inheritance

## Same resulting diagram, opposite starting point

Specialization and generalization produce the IDENTICAL final diagram shape. They differ only in which direction the designer reasoned: specialization splits an existing superclass; generalization merges existing entity types' common core. Notation and constraints apply identically either way.

Every occurrence of a subclass IS SIMULTANEOUSLY an occurrence of its superclass (every Manager row is also a Staff row) — the same “is-a” relationship object-oriented programming calls inheritance:

### Attribute inheritance

A subclass automatically has every attribute its superclass has, without redeclaring them. Manager has staffNo, name, position, salary, branchNo (inherited) plus bonus, mgmtLevel (its own).

### Relationship inheritance

A subclass automatically participates in every relationship its superclass participates in — Staff Manages PropertyForRent applies to SalesPersonnel and Manager too, no separate relationship drawn per subclass.

### Subclass-only relationships

A subclass may ALSO have its own additional relationships that other subclasses don't — e.g. only Manager participates in Manager Approves Expenditure.

# Constraints on Specialization / Generalization

Two independent yes/no questions fully describe how subclasses relate to each other and the whole superclass population:

## Disjoint (d)

An occurrence can belong to AT MOST ONE subclass. A Staff member is either SalesPersonnel or Manager, never both.

## Overlapping (o)

An occurrence CAN belong to more than one subclass simultaneously. A Person can be both a Student and an Employee at the same time.

## Total

Every superclass occurrence MUST belong to at least one subclass — no “plain” unspecialized occurrence exists.

## Partial

Some superclass occurrences MAY belong to no subclass at all, remaining a plain, unspecialized instance.

*Every specialization in this course is one of exactly four combinations of these two constraints.*

# The Four Disjoint/Overlapping × Total/Partial Combinations

## Disjoint, Total

Vehicle → Car, Van, Truck. Every vehicle is exactly one of the three; none is left unclassified.

## Disjoint, Partial

Staff → SalesPersonnel, Manager. Neither overlaps the other, but plenty of staff hold neither role.

## Overlapping, Total

Person → Student, Employee. Every person is at least one; some are both.

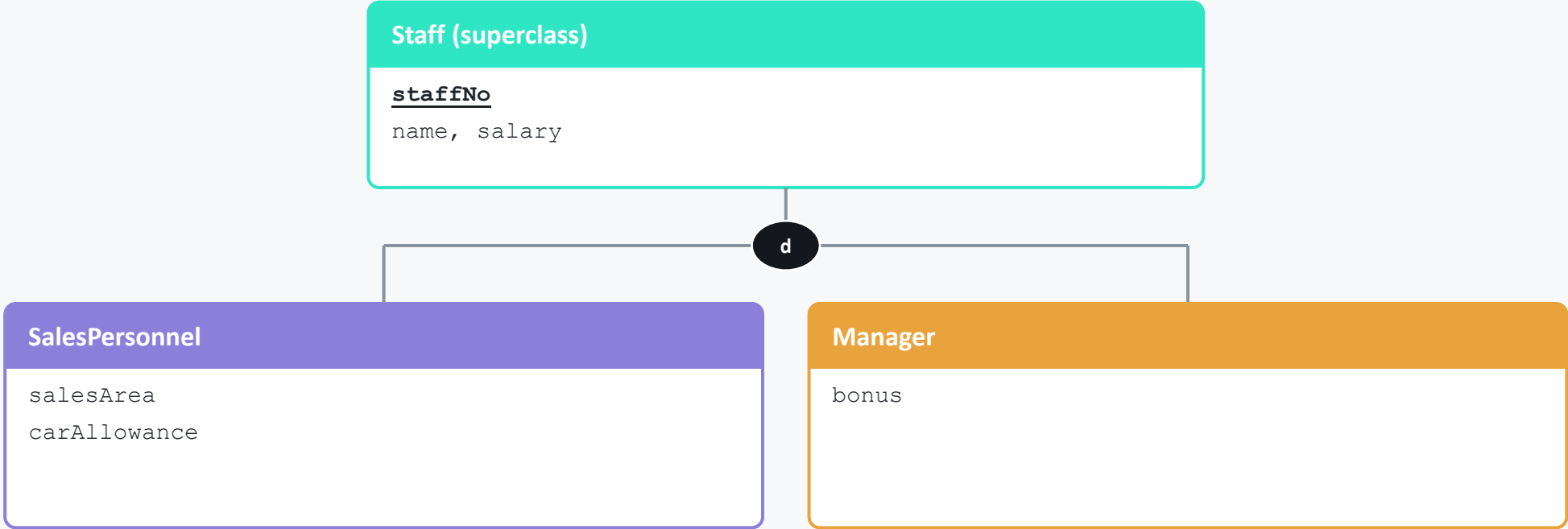
## Overlapping, Partial

Staff → CommitteeMember, Trainer. A staff member can hold both extra roles, one, or neither.

### Getting disjoint/overlapping wrong is a real, silent design bug

Declaring Staff → SalesPersonnel, Manager disjoint when a real staff member is later promoted to manager WHILE CONTINUING to hold sales targets forces the database to pick one, silently dropping a role that's still true. Always check against actual business rules — Lecture 16 shows this constraint changes which mapping strategy even works.

# Worked Example — Staff Specialization is Disjoint AND Partial

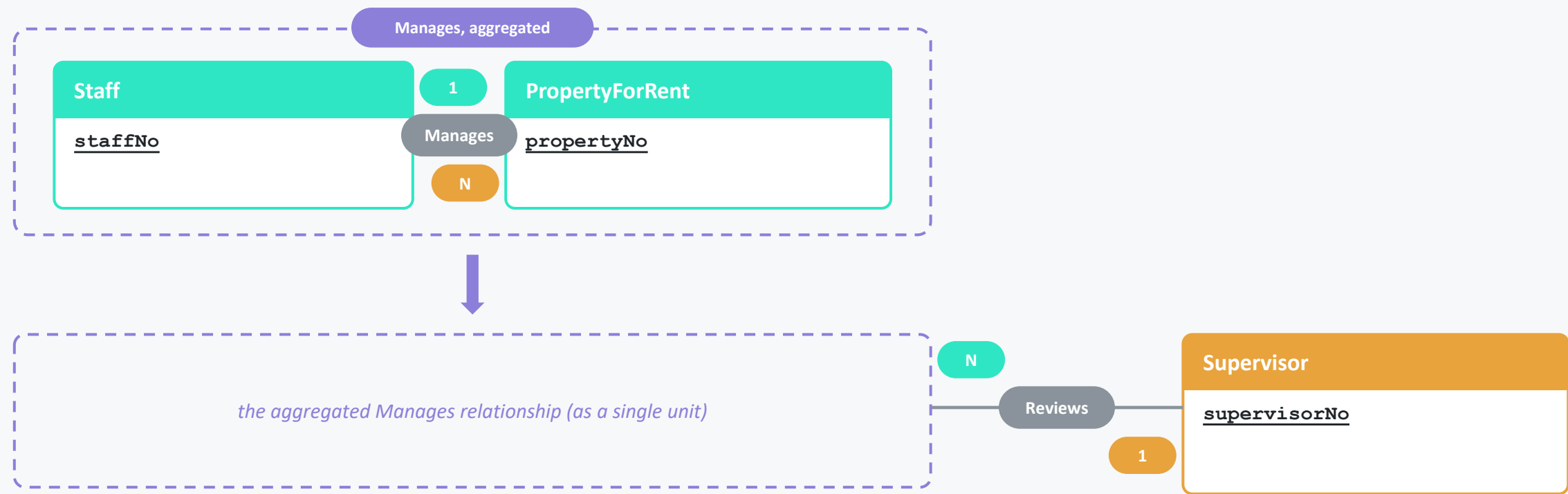


*disjoint · partial — an ordinary assistant belongs to neither subclass*

Partial means a plain “assistant” staff member — one who is neither in sales nor in management — is a perfectly valid Staff row that belongs to no subclass at all. Disjoint means no single staff member is simultaneously both a SalesPersonnel row and a Manager row.

# Aggregation

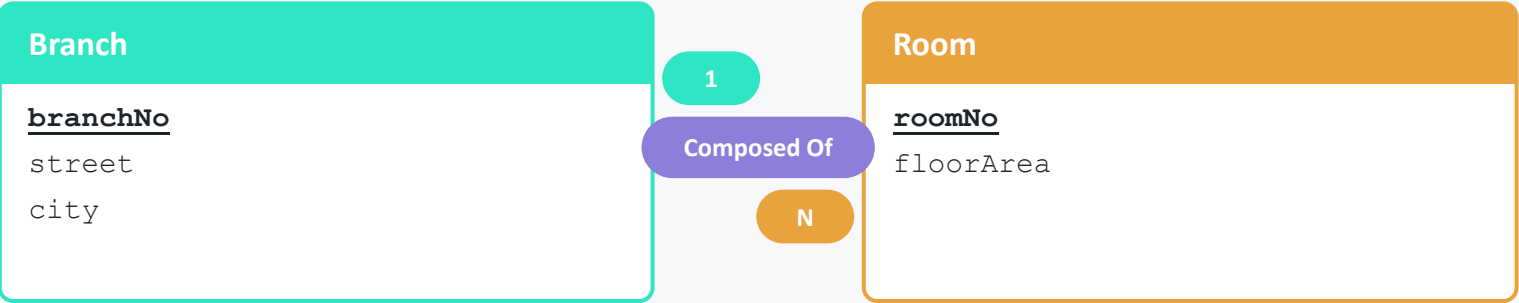
Aggregation models a “has-a” relationship AT THE RELATIONSHIP LEVEL — a relationship between two entity types must itself participate, as a whole, in a relationship with a third entity type. Plain ER can't connect a relationship to an entity; aggregation closes this gap.



A review is genuinely about the PAIRING (this staff member managing this property), not about either side alone — a supervisor might review the same staff member's management of one property differently from another.

# Composition

Composition is aggregation's stronger cousin: a whole-part relationship where the part has NO INDEPENDENT EXISTENCE apart from the whole — and a part typically belongs to exactly one whole at a time.

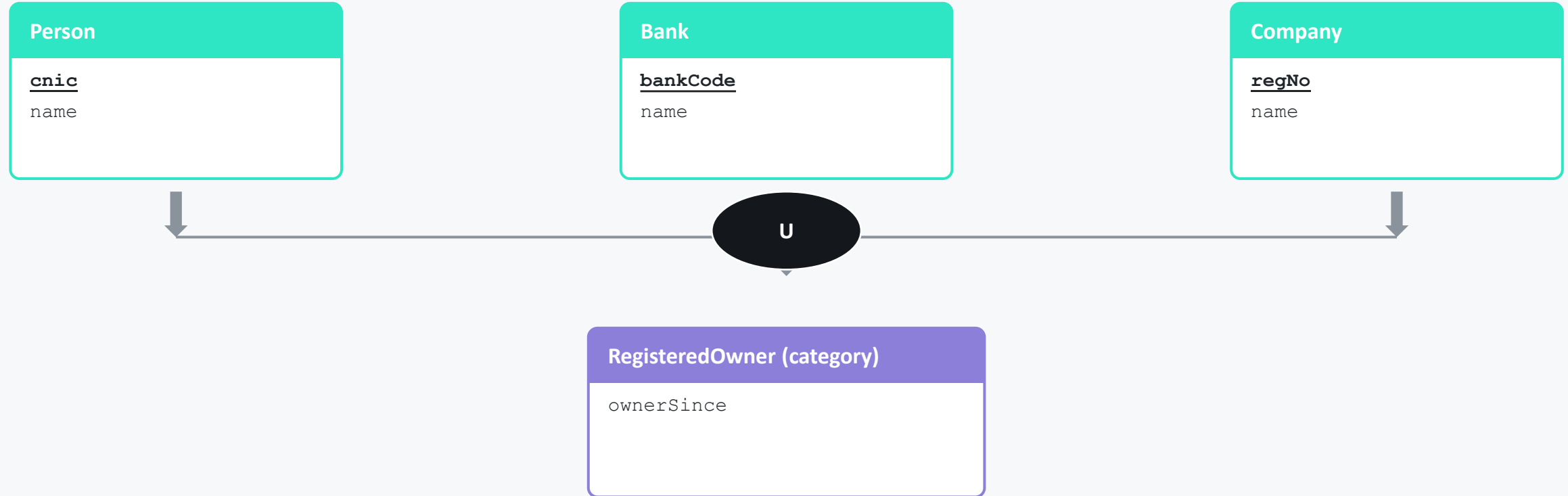


|                      | Aggregation                                | Composition                                    |
|----------------------|--------------------------------------------|------------------------------------------------|
| Existence dependency | The part can outlive the whole             | The part CANNOT exist without the whole        |
| Sharing              | A part can belong to more than one whole   | A part belongs to exactly one whole            |
| Deleting the whole   | Parts typically left in place / reassigned | Parts deleted along with the whole (cascading) |

*Composition almost always implies ON DELETE CASCADE on the resulting foreign key (Lecture 16); plain aggregation does not.*

# Categorization (Union Types)

A category's subclass represents a subset of the UNION of two or more distinct, otherwise unrelated superclasses — those parents don't share a common superclass of their own.



## Specialization vs. categorization, in one sentence

Ordinary specialization has subclasses sharing ONE common superclass. Categorization has a subclass drawn from the UNION of several different, unrelated superclasses — no single shared parent, only the category sitting above all of them.

# Key Takeaways

- Specialization reasons top-down (superclass → subclasses); generalization reasons bottom-up (subclasses → superclass). Both produce the same diagram shape.
- A subclass inherits every attribute and relationship of its superclass, and may add its own attributes and relationships.
- Two independent constraints classify every specialization: disjoint vs. overlapping, and total vs. partial — four combinations total, and getting the wrong one is a real design bug.
- Aggregation lets a relationship itself act as a single participant in another relationship — needed whenever a fact is genuinely about a pairing, not either entity alone.
- Composition is aggregation with a strong existence dependency; categorization (union types) models a subclass drawn from several unrelated superclasses.